Boxed: A Docker-Based Code-Execution Substrate for Autonomous Code-Generating Agents

Akshay Kumar
Independent Researcher
United States
akumar8@mt.iitr.ac.in

Abstract—Autonomous large language model (LLM) agents write, run, and revise code inside their reasoning loop, which puts the sandbox on the hot path of every task. Operators have three options today. Roll-your-own Docker is quick to set up but is usually deployed without a read-only root, capability drops, process caps, or egress control. Hosted sandbox services are hardened but move the operator's code and data to a third party. Research virtual machine monitors isolate well but are built for a boot-once serverless model. Boxed is an open-source substrate for agentic code execution: a self-hosted Go control plane, a 1.32 MiB Rust in-sandbox agent that streams stdout, stderr, and files over JSON-RPC 2.0 as they appear, and a driver interface whose four lifecycle methods are meant to admit backends beyond the Docker driver that exists today. We measure that driver against the Docker Engine API called directly on the same host, image, and command. Over 5 runs of 200 lifecycles, Boxed's median create+exec+destroy latency is 178 ms; raw Docker with the same hardened configuration takes 153 ms, so the control plane and agent cost 25 ms (bootstrap 95% CI 23–27 ms). The hardening itself makes raw Docker 63 ms faster than stock defaults. A twelve-vector escape probe scored by post-conditions read from the host denies 12 of 12 vectors. Scoring the same run with the signature classifier from the earlier version of this work changes 2 verdicts, which measures how much that method misreports. Boxed, the harness, and the raw traces are MIT-licensed.

Index Terms—sandboxing, code execution isolation, large language models, autonomous agents, containers, Docker, JSON-RPC

I. Introduction

Autonomous code-generating agents arrived in force between 2024 and 2026, from Yang et al.'s SWE-agent [1] and the AutoGPT-style harnesses to products such as Cursor, Devin, and Claude Code, and they changed what "running untrusted code" means in practice. Executable code has been argued to be a better action space for an agent than text or JSON [2]. An agent built that way emits dozens of short programs per task: it writes a snippet, runs it, reads stdout, reads back a file it produced, and tries again. The sandbox sits inside that loop, and its latency is paid on every iteration.

An operator who wants that loop hosted has three options today, and each gives something up.

Roll-your-own Docker is the common choice. It is quick to set up and is routinely shipped without seccomp profiles, egress filters, a read-only root filesystem, or process and memory caps, which leaves the host one rm -rf / away from compromise [3]. Measurement studies of deployed containers report the same pattern at scale [4], and cgroup-based resource isolation can itself be evaded [5].

Hosted sandbox services (E2B [6], Modal Sandbox [7], Cloudflare Sandbox [8]) ship hardened isolation. In exchange, every byte of generated code, execution output, and intermediate artifact leaves the operator's infrastructure for a third party, which some deployment and data-handling rules do not allow.

Research VMMs such as Agache et al.'s Firecracker [9] have the right primitives. They are built for AWS Lambda's boot-once, amortized invocation model, and they have no in-guest agent to stream artifacts back to the caller.

What agent workloads need is a substrate: an isolation backend chosen at deployment time, an agent inside the sandbox that streams standard output and on-disk artifacts as they appear, and a deployment the operator runs, with the control plane and the workload data under their control.

A. Problem Statement

An autonomous coding agent runs untrusted, machine-generated code many times in a single task, and three things have to hold at once. The isolation boundary has to contain code nobody reviewed. Per-execution latency has to stay low enough that the sandbox does not dominate the agent's loop. And the operator has to keep control of the keys and the data, because agent workloads carry proprietary source and customer records into the sandbox as a matter of course.

No existing substrate satisfies all three. Hosted services meet the first two, but they put a third party inside the trust boundary and give the operator no say over the isolation backend. Self-hosted container runtimes leave the operator in charge but tie the control plane to one isolation technology, so a stronger boundary later means rewriting the orchestration layer. Boxed targets that gap: keep the control plane, the keys, and the data with the operator, and make the isolation backend a deployment-time choice.

Two questions follow, and Section VI answers them by measurement. What do the control plane and agent cost over calling Docker directly on the same host, image, and command? And once the Docker driver applies the hardening that roll-your-own deployments skip, which of a standard set of container-escape vectors does it stop, judged by the state left after each attempt and not by strings in the attempt's output?

B. Objectives

- Define an isolation interface narrow enough that container runtimes, microVMs, and WebAssembly sandboxes could be interchanged behind it, so that the security boundary can be strengthened without changing the control plane.
- Deliver execution output incrementally, so that an agent can react to partial results before process exit, and return files produced during execution as structured events on the same channel.
- Support self-hosted deployment without requiring a Boxed-operated service to receive workload data.
- Quantify the cost and the limits of the resulting design on commodity hardware against a raw-Docker baseline, and report the isolation failures the configuration does not prevent.

C. Contributions

- A Go driver interface whose four lifecycle methods (Create, Start, Stop, Connect) are the contract a backend has to satisfy, so that the isolation backend becomes a deployment-time setting. The release implements one backend, Docker. No microVM or Wasm backend exists yet, and this paper claims only what the Docker driver does.
- A Rust in-sandbox agent that streams stdout and stderr chunks as discrete JSON-RPC events instead of buffering until process exit, mirrors files written under a watched directory back to the caller as artifact events, and reports the child's real exit status.
- A self-hosted control plane authenticated by an operator-configured API key. It authenticates one instance and does not manage credentials or tenants.
- A released benchmark harness and campaign: lifecycle latency against raw Docker measured through the Engine API on the same host; throughput under concurrency with repeated sweeps and confidence intervals; idle per-sandbox overhead; a twelve-vector escape probe scored by post-condition checks read from the host; and an end-to-end agent trace on official HumanEval tasks with the model, prompts, token counts, and exit codes recorded. The raw CSV and JSONL traces ship with the source, next to the earlier pre-hardening traces for audit.
- A comparison of two ways to score an escape probe. The signature classifier from the earlier version of this work and the post-condition checks used here are applied to the same attempts, and the paper reports where they disagree.

II. Background and Related Work

A. Container-based isolation

Linux containers, built on namespaces, cgroups, and seccomp, underpin nearly every development sandbox. They start in roughly 50–200 ms when images are warm, which is why they dominate workloads that create and destroy environments often. The cost is that containers share the host kernel, a much larger attack surface than a hardware boundary. CVE-2019-5736 [3] is the standard reminder that one bug in runc turns a container into the host. Lin et al. [4] ran 88 of 223 collected exploits inside a default-configured container: 50 succeeded, capabilities, seccomp, and MAC stopped more privilege escalations than namespaces and cgroups did, and 11 exploits still broke isolation. That ordering is why the hardening in Section V-B starts with the capability drop. Hardened runtimes such as gVisor [10] put a userspace kernel between the workload and the host. Young et al. [11] measure its syscall-interception cost at 2–5×. Anjali et al. [12] compare Linux containers, gVisor, and Firecracker on CPU, network, memory, and file system microbenchmarks and trace the kernel code each executes; both gVisor and Firecracker run substantially more kernel code than native Linux, and neither wins on every workload. Docker can select gVisor as a runtime (–runtime=runsc), which makes it the cheapest stronger boundary for a Docker-based substrate (Section VII).

nsjail, bubblewrap, and firejail apply the same namespace, seccomp, and cgroup primitives to a single process, with no image and no daemon. For an operator who does not need Docker's image ecosystem they are the natural comparison. Boxed uses Docker for two reasons: agent workloads are specified as images (a Python runtime, a toolchain, a set of packages), and the Engine API provides the exec-attach stream the in-sandbox agent runs over. A process-sandbox backend behind the same interface is possible but is not evaluated here.

B. Lightweight virtual machines

Agache et al. [9] demonstrated that a stripped-down KVM VMM can boot a guest kernel in ∼125 ms with <5 MiB of memory overhead per VM, and Firecracker now powers AWS Lambda. Kata Containers [13] and Manco et al. [14] pursue the same hypervisor-as-isolation thesis from complementary angles. These systems optimize boot cost and steady-state memory for a serverless pattern: boot once, run an event loop, spread the startup cost over many invocations. A coding agent does the opposite. It creates a sandbox, runs one short program, pulls out the results, and destroys the sandbox, many times per task, so the cost that matters is the whole lifecycle and not cold start alone.

C. Unikernels and WebAssembly

Madhavapeddy et al. [15] compile the application together with a library operating system into a single boot image, minimizing the trusted computing base at the cost of application-specific compilation, which rules out arbitrary dynamically generated binaries. WebAssembly [16] and WASI [17] make the guest portable bytecode with capability-based imports and the sandbox a runtime in the host process; microsecond instantiation is what a create-heavy agent workload wants, but agents emit unmodified Python scripts, shell commands, and native binaries that assume a POSIX environment, which the WASI toolchain does not yet run unmodified.

D. Agent-oriented sandbox platforms

Several products now target LLM agents specifically. E2B [6] runs Firecracker microVMs behind a SaaS API and reports about 150 ms cold start; Modal Sandbox [7] layers gVisor over Firecracker; Daytona [18] ships an AGPL, Docker-backed, self-hosted development environment; Cloudflare Sandbox [8] reuses V8 isolates for about 5 ms cold start and limits execution to JavaScript and Wasm. Among research systems, OpenHands [19] executes every agent action in a Docker container through a REST action-execution API that runs inside it. That is the closest design to Boxed's driver-plus-agent split, but OpenHands is an agent framework with a sandbox inside it and has no backend interface. The vendors measured their cold-start figures under conditions they do not disclose; Table I repeats them as reported, and they are not comparable with our end-to-end lifecycle numbers. Two limits hold across the category: the platform fixes the isolation mechanism, and apart from Daytona and OpenHands, the platforms are mainly hosted services.

E. Evaluating LLM-generated code execution

Chen et al. [20] made single-function code generation a standard benchmark. Yang et al. [1] extended it to agent-mediated software engineering, where the agent has to run the code it writes and refine it. Our agent trace (Section VI-F) follows that regime on official HumanEval tasks: the model emits a candidate Python function, Boxed runs it against the task's published test, and the harness records the process exit status. Splitting end-to-end wall-clock into model inference and sandbox lifecycle shows how much of an agent's latency a substrate can affect at all.

III. Threat Model

The threat model assumes an adversary with full control over the code submitted to a sandbox: they may craft arbitrary Linux binaries, issue any syscall sequence, and attempt network egress to any destination. The adversary does not control the control plane, the driver implementation, or the host kernel; these remain trusted. Fig. 1 shows the resulting trust boundary.

The design aims to prevent four classes of attack.

T1, host filesystem access. Read or write access to the host filesystem outside the sandbox's ephemeral root, whether through path manipulation, symbolic links, or mount operations.

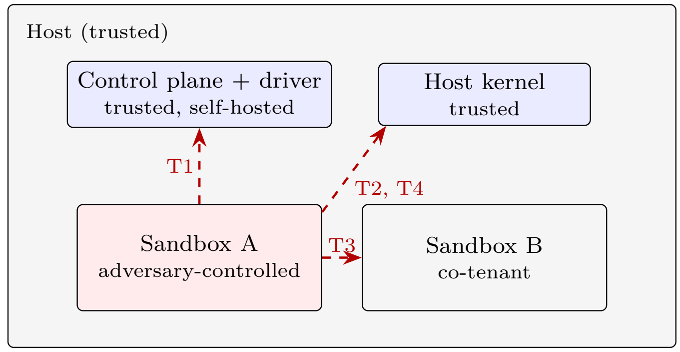

Fig. 1. Trust boundary. Adversary-controlled code executes inside a sandbox; the control plane, driver, and host kernel are trusted. Dashed arrows are the four threat classes the substrate must resist. Side channels between co-tenants on shared hardware are out of scope.

T2, namespace escalation. Escape from the allocated network or process namespace into the host's network stack or process tree, or tampering with the in-sandbox agent from a workload process that shares its namespace.

T3, co-tenant exfiltration. Access to credentials, artifacts, or data belonging to a concurrent sandbox on the same host.

T4, resource exhaustion. Denial of service against the host through fork bombs, memory exhaustion, or other unbounded resource consumption.

Side channels through shared hardware (cache timing, speculative execution) are out of scope; defending against them means per-customer hardware partitioning, which is the operator's decision. The escape probe in Section VI-E covers T1, T2, and T4 directly. It covers T3 only through the network vectors, since the sandbox has no route to a co-tenant except the network, and no vector tries a cross-sandbox read through the host. The limitations record that gap.

IV. Design

Boxed has three components and one narrow interface (Fig. 2): a stateless control plane, a driver layer, and an in-sandbox agent. Orchestration is kept apart from the isolation mechanism so that one control plane could drive Docker on a laptop and a microVM backend in production without change. Today only the Docker half of that exists.

A. Driver Interface

The central abstraction is one Go interface. In the current source it has twelve methods; eight are helpers for file injection, listing, health, and naming that the SDK and control plane use. The four lifecycle methods below are the minimum a backend has to implement, and the control plane's sandbox path depends on nothing else:

```
type Driver interface {
 Create(ctx context.Context,
```

TABLE I
Agent-oriented sandbox runtimes (data as of 2026-05). Cold-start figures for systems other than Boxed are vendor claims measured under undisclosed conditions and are reproduced as reported, not validated by us; the Boxed figure is the end-to-end create+exec+destroy median of Section VI-B and includes an exec, which the vendor figures may not.

System	Isolation	Cold start	Backend selectable	Self-host	License	First-class artifacts
Boxed (this work)	Docker (runc)	178 ms (measured)	interface only*	yes	MIT	yes
E2B [6]	Firecracker	∼150 ms (claim)	no	limited	Apache-2.0	yes
Modal Sandbox [7]	gVisor + Firecracker	∼1.5 s (claim)	no	no	proprietary	yes
Daytona [18]	Docker	not reported	no	yes	AGPL	no
Cloudflare Sandbox [8]	V8 isolates	∼5 ms (claim)	no	no	proprietary	limited

*The driver interface is designed for other backends; Docker is the only implementation in this release.

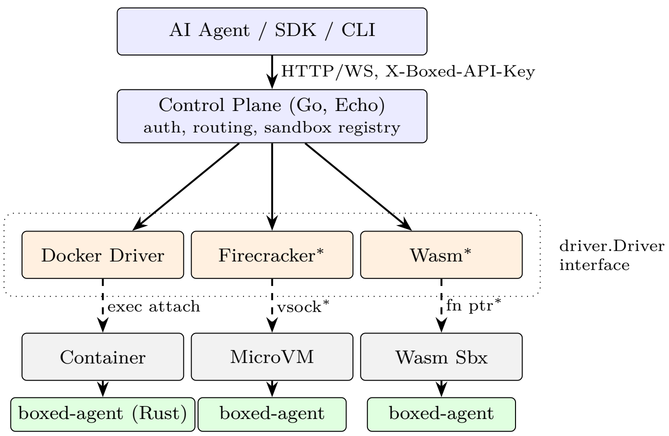

Fig. 2. Boxed architecture. The control plane drives one Driver interface. The Docker driver is the only implementation; the Firecracker and Wasm boxes show where a microVM or Wasm backend would attach. A common Rust agent runs inside every sandbox and speaks JSON-RPC over whatever byte stream the driver supplies. *Not implemented; shown as a design target only.

```
cfg SandboxConfig) (string, error)
Start(ctx context.Context, id string) error
Stop(ctx context.Context, id string) error
Connect(ctx context.Context,
id string) (io.ReadWriteCloser, error)
// ... file, listing, and health helpers
}
```

Create builds a sandbox from the configuration and returns an identifier. Start and Stop bound its lifetime. Connect returns a raw byte stream that the control plane uses to talk to the in-sandbox agent: for the Docker driver it is a Docker exec attach, for a microVM driver it would be a vsock connection, and for a Wasm driver a host-side function. The control plane does not know which it has. Whether four methods are enough for a microVM backend has not been tested, since no such backend has been written; it is listed as future work.

B. Control Plane

The control plane is a stateless HTTP and WebSocket server on the Echo framework. It maps POST /v1/sandbox requests through the configured driver, and two of its choices matter for agent workloads.

create returns as soon as the runtime reports that the sandbox process launched (for Docker, when ContainerStart acknowledges), with status: ready. The client never polls a status endpoint, so the control plane carries no polling load. The first exec after create absorbs the wait for the agent's JSON-RPC loop to come up. An agent task may create dozens of sandboxes, and one round trip saved per sandbox adds up.

Authentication is a static, operator-configured API-key header (X-Boxed-API-Key). Boxed does not issue, rotate, or store keys. This authenticates one instance. It does not do multi-tenant identity, key management, or compliance. In a self-hosted deployment, generated code, execution output, and intermediate files stay on infrastructure the operator controls.

C. In-Sandbox Agent

boxed-agent runs inside every sandbox. It is one Rust binary built on tokio, serde, and notify (1.32 MiB stripped; it links the C library dynamically and needs nothing else at runtime), and it speaks JSON-RPC 2.0 as newline-delimited JSON over whatever byte stream the driver supplies.

On exec the agent spawns the command as a child process and streams stdout and stderr chunks back as separate RPC events as they arrive, so a model can react to partial output or an early error before the program finishes. When the child exits the agent reports its exit status. An earlier release reported zero unconditionally; the fix is in the current source, and Section VI-F covers what it did to the agent-trace results. Alongside, an inotify watcher monitors the artifact directory (/output). Any file written there is emitted as an artifact event with the file's MIME type and base64 contents, so an agent that asks Python to write plot.png gets the bytes pushed to it, with no second round trip and no polling.

D. Artifact Protocol

Files are first-class outputs because much of what a model asks code to do shows up as a side effect on disk and never reaches the console.

The current release streams them inline: the watcher reads the file, guesses its MIME type, base64-encodes it, and emits it on the same channel as stdout and stderr. A 10 MiB cap bounds the memory and serialization cost; larger files are skipped with a logged warning. The wire protocol reserves a url field on the artifact event for an out-of-band path, in which the agent would PUT the file to a pre-signed URL and return a reference, and that path is not implemented. We say so plainly because a substrate that silently drops large artifacts is a correctness hazard for any workload that emits checkpoints or datasets.

Both the protocol and the agent sit above the driver interface, so artifact semantics do not depend on the backend.

V. Implementation

The release is 2,432 lines of Go in cmd and internal (control plane, Docker driver, CLI) and 918 lines of Rust for the agent. The control-plane binary is 12.0 MiB; the agent binary is 1.32 MiB stripped. Two of the details below are easy to assume wrongly, so they are stated exactly.

A. Agent injection and container lifecycle

The Docker driver bind-mounts the host's boxed-agent binary into the container read-only at create time, so a stock base image (python:3.10-slim) can be reused across sandboxes with no Boxed-specific image registry to maintain. The image root filesystem is read-only. Separate tmpfs mounts back /tmp, /output, and the configured working directory, so user-written data is ephemeral and never touches the host filesystem.

The container's own PID 1 is a tail -f /dev/null placeholder whose only job is to keep the namespace alive. Connect launches boxed-agent as a Docker exec, and the attached stream carries JSON-RPC. This matters for security: the agent is a child of the container's init, so any workload process in the sandbox's PID namespace can see it, which is why the ptrace vector in Section VI-E targets the agent. Under a microVM backend the agent would be the guest init and talk over vsock.

B. Resource and network model

Every sandbox is created with a CPU quota (NanoCPUs), a memory cgroup, and a PidsLimit of 256. The container root is read-only, all Linux capabilities are dropped, and no-new-privileges is set. Docker's default seccomp profile applies; Boxed does not yet ship its own. The resource defaults are 1.0 core and 512 MiB, with ceilings of 4.0 cores and 8 GiB. A TTL goroutine stops the sandbox when its timeout expires.

The Docker backend always uses Docker's none network, so only loopback exists. Requests that enable networking or name an allow-list are rejected until Boxed has an enforceable per-sandbox egress policy. This fails closed. It does not claim that Docker's default bridge would be an adequate egress control.

The pre-hardening release lacked all of these settings. The May 2026 traces kept under bench/results/legacy-2026-05-docker-desktop were taken without the read-only root, the capability drop, the PID limit, or the network restriction, and with the exit-status defect in the agent. Every number in Section VI comes from the hardened build.

VI. Evaluation

This section answers the two questions from Section I: what the control plane and agent cost over calling Docker directly, and which escape vectors the hardened driver stops under post-condition scoring. It also reports throughput under concurrency, idle per-sandbox overhead, and an end-to-end agent trace. The harness, the campaign runner, and every raw trace are in the repository, and every number below expands from a macro that bench/-analyze/stats.py generates from those traces.

A. Setup

All measurements were taken on a MacBook Pro with an Apple M1 Pro SoC and 16 GB of unified memory running macOS 26.5, with Docker Engine 29.5 (linux/arm64, kernel 6.8, cgroup v2, seccomp and AppArmor enabled) inside a colima virtual machine using Apple's Virtualization framework, provisioned with 4 vCPUs and 8 GiB. The python:3.10-slim image was pre-pulled. The machine ran its usual background processes (editor, browser) throughout and was not quiesced; we report dispersion across repeated runs so that the contention shows. Unless noted otherwise each lifecycle runs a synthetic print('ok') workload, which keeps application compute out of the substrate measurement. The Boxed control plane was built from the tagged release with Go 1.24 and the agent with Rust 1.80.

B. Lifecycle latency against raw Docker

The substrate's cost only means something next to what Docker costs on the same host, so three configurations ran the same create→exec→destroy sequence. Raw Docker, stock defaults calls the Engine API with docker run's default host configuration. Raw Docker, hardened calls the Engine API with exactly the host configuration Boxed's driver applies (read-only root, CapDrop ALL, no-new-privileges, PID limit, network none, 1 CPU, 512 MiB, tmpfs mounts) and has no control plane or agent. Boxed goes through the HTTP API. The raw configurations run python3 -c "print('ok')" through a Docker exec; Boxed runs the same command through the in-sandbox agent. Each configuration ran 5 runs of 200 sequential lifecycles, interleaved so that drift on the host hits all three alike.

Table II and Fig. 3 give the result. Boxed's median end-to-end lifecycle is 178 ms (bootstrap 95% CI 177–179 ms; per-run medians 172 ms to 190 ms). The interquartile range is 33 ms, and the tail reaches 327 ms at p95 and 578 ms at p99. Raw Docker with the same hardened configuration takes 153 ms. The difference of 25 ms (95% CI on the difference of medians 23–27 ms; 16% of the raw cost) is what the HTTP control plane, the agent launch over exec

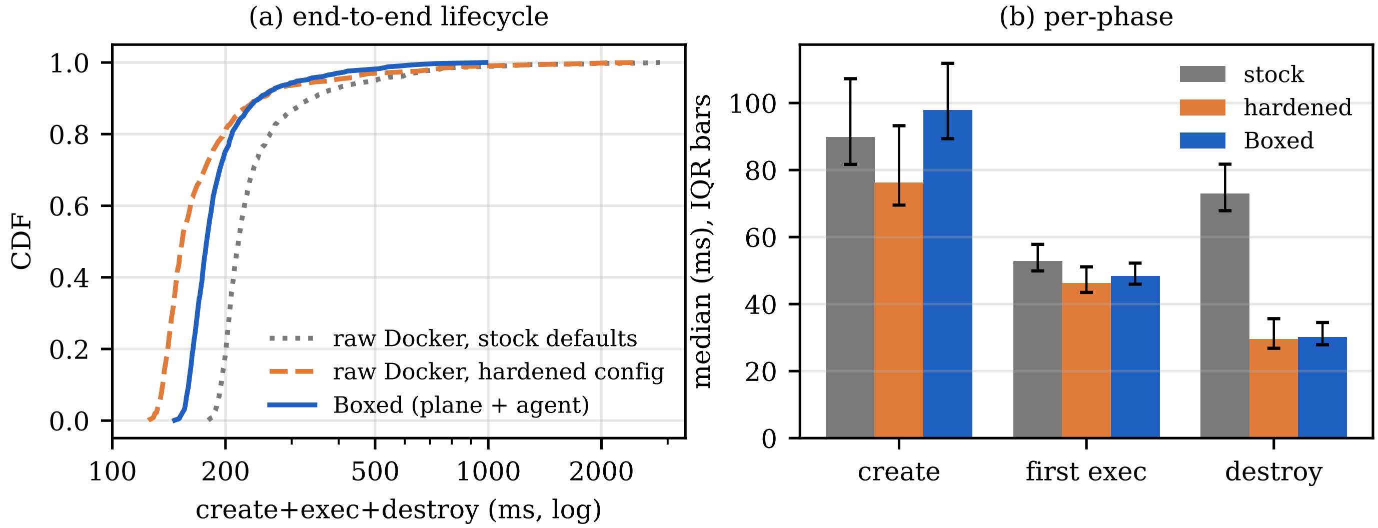

Fig. 3. Lifecycle latency, 5 runs × 200 lifecycles per configuration. (a) CDF of end-to-end create+exec+destroy. (b) Median per phase with interquartile bars. The difference between the Boxed and hardened raw-Docker bars is the cost of the control plane and the in-sandbox agent.

TABLE II
Lifecycle latency, medians over 5 runs × 200 sequential create→exec→destroy cycles, same host, image, and command. Raw Docker rows call the Engine API directly with no control plane and no agent.

Configuration	create	exec	destroy	total	p95
Raw Docker, stock defaults	90	53	73	216	499
Raw Docker, Boxed's hardened config	76	46	29	153	379
Boxed (control plane + agent)	98	48	30	178	327

All values in ms.

TABLE III
Throughput sweep: 80 create→destroy lifecycles per level, 10 independent sweeps. Mean sandboxes/s with 95% confidence interval (Student's t) and the range across sweeps. 0 errors in 4800 lifecycles.

Concurrency c	Sandboxes/s (mean)	95% CI	Range
1	6.98	±0.42	5.70–7.59
2	11.44	±0.97	9.16–13.49
4	14.32	±2.25	6.34–16.83
8	16.43	±2.98	9.99–21.29
16	17.19	±2.68	12.23–22.77
32	17.14	±2.41	11.73–20.94

attach, and the JSON-RPC exec path add together: 22 ms on create, 2 ms on the first exec, and 1 ms on destroy. It buys an HTTP API that any language can call, an agent that streams output and files as events, and a create that takes one round trip. An operator who already drives the Engine API from Go and does not need streaming artifacts would not pay it.

The two raw-Docker rows compare the hardening against stock defaults. Boxed's hardened configuration is 63 ms faster end to end than docker run's defaults. The stock configuration attaches the container to the default bridge network, and setting up and tearing down that attachment costs more than the tmpfs mounts and the capability drop do; with network none that work is gone. On this host the hardening that roll-your-own deployments skip does not cost latency.

C. Throughput under concurrency

Client concurrency was swept over $c \in \{1, 2, 4, 8, 16, 32\}$ with 80 create→destroy lifecycles per level, and the whole sweep was repeated 10 times (Table III, Fig. 4). Throughput rises from 7.0 sandboxes/s at c=1 to a plateau of 17.2 (±2.7) at c=16 and 17.1 at c=32; the last three levels sit inside one another's confidence intervals. The VM has four vCPUs, and a single Docker daemon serializes every container operation, which is the plateau. The range column is the reason for the repeats. A single sweep at c=4 or c=8 can land anywhere in a two-to-one band, and the earlier version of this work read a shape into one sweep per level (a dip at c=16, a collapse at c=32) that these repeats do not reproduce.

D. Idle per-sandbox overhead

Each of 30 idle sandboxes was sampled five seconds after create returned. Median working set, as docker stats reports it from the container cgroup, is 0.43 MiB (p95 0.67 MiB, maximum 0.68 MiB); median CPU is 0.01%. This is cgroup accounting inside the VM and not a process-level measurement on a native host, so we take it as an order of magnitude. It fits the design: the agent is a stripped Rust binary that holds no buffers and waits on one tokio event loop for a JSON-RPC frame, whereas a Python or Node agent would carry an interpreter and a module cache while idle.

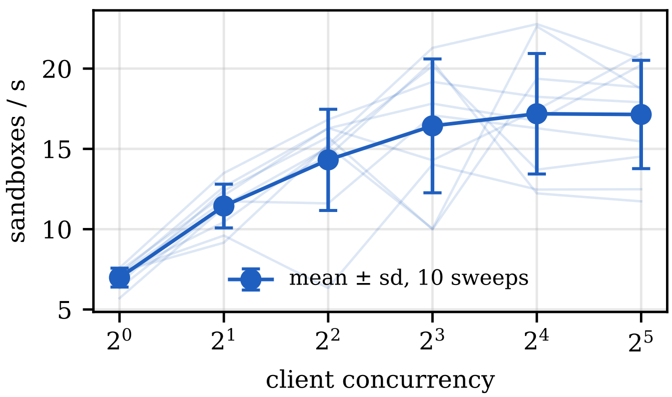

Fig. 4. Sustained sandbox-creation rate against client concurrency. Thin lines are the 10 individual sweeps; markers are the mean with one standard deviation.

E. Escape probe with post-condition checks

The hardened driver was exercised with twelve attempts drawn from common container-escape patterns: mounting a filesystem, reading the host through /proc/1/root, reaching the Docker or containerd socket, loading a kernel module, opening a raw socket, creating a new user and PID namespace, egress to an RFC1918 address, egress to the link-local metadata endpoint, a fork storm, a memory bomb that touches every page it allocates, a write to the root filesystem, and PTRACE_ATTACH on the in-sandbox agent. Each attempt runs in a fresh sandbox, and the whole suite ran 3 times.

Each vector gets two verdicts on the same attempt. The signature verdict is the method from the earlier version of this work: capture stdout and stderr and grep for a denial string. The post-condition verdict looks at state after the attempt: whether a mount appeared in /proc/mounts, whether the written file exists, the errno the syscall returned, which device and inode /proc/1/root resolves to, and, for the resource vectors, the cgroup's pids.events and memory.events counters read on the host, outside the sandbox. Table IV gives both verdicts and the evidence for every vector.

The post-condition checks classify 12 of 12 vectors as denied (100%), with 0 undenied and 0 harness errors, and the verdicts were identical across the 3 runs.

The evidence column settles the cases the signature method could not. The cgroup kills the memory bomb, and the host's oom_kill counter records it, but the process prints nothing before SIGKILL, so the signature method took that silence for a successful attack. The fork storm gets exactly as many forks as the PID limit allows and then EAGAIN, and the host's pids.events counter shows the limit engaging. /proc/1/root resolves to the same device and inode as the container's own root, so listing it crosses nothing. The Docker socket check finds no socket, and that absence is the denial.

The network denials are now Boxed's own. In the pre-hardening configuration the sandbox sat on Docker's bridge, and the RFC1918 and metadata denials were plausibly Docker Desktop's NAT topology. With network none the sandbox has a loopback interface and nothing else, as the evidence column shows, so there is no route to deny.

The ptrace vector, which the pre-hardening driver let through, is denied on the hardened driver because CapDrop ALL removes CAP_SYS_PTRACE and the kernel's Yama scope of 1 limits attachment to descendants, and the agent is a sibling of the workload. Both parts are needed: a host with Yama disabled would need the seccomp profile Boxed does not yet ship. The module-load vector is likewise scored by calling init_module directly. The earlier probe invoked a modprobe binary that the image does not contain, so its "not found" verdict measured the image and said nothing about the boundary.

The signature and post-condition verdicts disagree on 2 of 12 vectors on the same run. In each case the signature method reported an undenied attempt that the post-condition evidence shows was stopped. A probe scored by strings in captured output therefore understates coverage on exactly the vectors where enforcement kills or silences the process, which is how the resource limits work. The earlier version of this work published the signature verdicts, so the correction belongs in the record.

This is a behavioural probe over twelve vectors and does not prove isolation. It does not exercise kernel vulnerabilities, it runs one image, and it has no cross-sandbox read (T3). None of the twelve attempts escaped the host.

F. End-to-end agent trace

A code-generation loop drove Boxed over the first 20 tasks of the official HumanEval release [20] in file order (HumanEval/0 onward), 3 times, for 60 task executions. For each task the model (claude-opus-5, default adaptive thinking, 4096 max output tokens, called through the Anthropic Messages API with no fallback model, so every completion comes from the named model) gets the task's signature and docstring and returns the function. Boxed runs the prompt, the completion, and the task's published check function in a fresh sandbox, and the process exit status decides pass or fail. A wrong candidate pushed through the same path exits 1 with an AssertionError; the earlier release would have reported it as passing. The model identifier, timestamp, prompt hash, token counts, stop reason, raw completion, executed program, and captured output for every task are in agent_trace.jsonl next to each CSV.

60 of 60 task executions passed (100%; stop reasons end_turn: 60). Median end-to-end wall-clock was 3.95 s: the model call took 3.42 s and the sandbox lifecycle 453 ms (per-repetition medians 436–504 ms; Fig. 5). Summed over all executions, model inference is 85% of wall-clock (80–88% across repetitions) and the sandbox 15%. Sandbox

TABLE IV
Escape probe on the hardened Docker driver, 12 vectors, 3 independent runs (verdicts identical across runs). Signature is the v1 classifier (grep for a denial string in captured output); Post-condition checks state after the attempt, on the host where possible. The paper reports the post-condition column.

Vector	Threat	Exit	Signature	Post-condition	Evidence
mount_host	T1, T2	0	denied	denied	mounts_on_/mnt=0
proc1_root	T1	0	denied	denied	proc1_root_dev_ino=42:1193209 root_dev_ino=42:1193209 pid1=tail
docker_sock	T2, T3	0	undenied	denied	sockets_found=0 api_reachable=False
modprobe	T2	0	denied	denied	init_module errno=1(Operation not permitted)
raw_socket	T2	0	denied	denied	errno=1(Operation not permitted)
setns	T2	0	denied	denied	unshare rc=1 unshare: unshare failed: Operation not permitted
rfc1918	T3	0	denied	denied	ifaces=lo
imds	T3	0	denied	denied	ifaces=lo
fork_bomb	T4	0	denied	denied	forked=250 EAGAIN after 250 forks host:pids.max=256 pids.events=max 1
mem_bomb	T4	−1	undenied	denied	exit=-1 host:memory.max=536870912 oom_kill 1
ro_root	T1	0	denied	denied	/etc:absent /usr/bin:absent
ptrace_agent	T2, T3	0	denied	denied	PTRACE_ATTACH pid=7 errno=1(Operation not permitted) yama=1

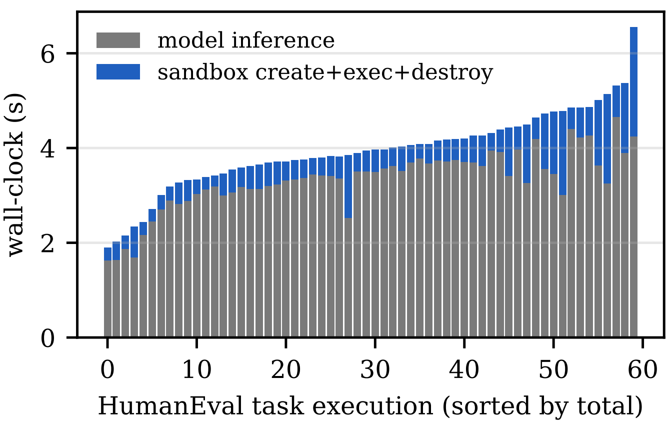

Fig. 5. Model inference versus sandbox time per task execution over 60 HumanEval task executions (3 repetitions of 20 tasks); a red mark would denote a failed task.

create in this loop had a median of 279 ms (268–328 ms across repetitions), against 98 ms in the tight lifecycle loop of Section VI-B. During all three repetitions the host's one-minute load average was between 13 and 17 on eight cores, from activity unrelated to the experiment; the lifecycle campaign ran under lighter load, and the recorded load is kept next to the traces. So the gap is an upper bound on what an idle interval between creates costs, and on a quieter host the model's share would be larger. The pass rate measures the model. For the substrate, the trace shows that the exit status now carries the test outcome, and how large a share of an agent iteration the sandbox takes with a current frontier model.

G. Threats to validity

All measurements come from one laptop, and Docker ran inside a virtual machine, so absolute latencies include a hypervisor and virtio path that a native Linux host would not pay. The earlier campaign on Docker Desktop's LinuxKit VM measured 303 ms for the same lifecycle on the unhardened driver, and the two VM stacks cannot be compared. The comparison the paper rests on, Boxed against raw Docker, holds the host, VM, image, and command fixed, and the interleaved runs give a dispersion estimate; that comparison survives a change of platform in a way the absolute numbers do not. The host was not quiesced. The escape probe is twelve vectors on one image, and it tests configuration; it does not test kernel vulnerabilities. Throughput lifecycles are create→destroy without an exec, so they measure the daemon's container churn rate and not the latency an agent sees. The figures for other systems in Table I are vendor claims we did not reproduce. The agent trace uses one model and twenty tasks, three times, on a loaded host.

VII. Discussion

A. Hardening status

Table V maps each gap the pre-hardening probe exposed to its status in the current source. The read-only root, capability drop, PID limit, privilege-escalation block, and network restriction are in place, and the post-condition probe confirms each of them. Two items remain: a custom seccomp profile, so that the ptrace denial stops depending on the host's Yama setting, and a per-sandbox egress allow-list, until which networked execution is refused. The upload path for artifacts above the inline cap is also unimplemented.

B. gVisor as the next boundary

Because Docker accepts runsc as a container runtime, the cheapest stronger boundary for this substrate is a one-field change to the driver's host configuration, with the same image, agent, and control plane. It was not measured here because gVisor is not supported inside the macOS virtual machine used for the campaign.

C. Pool-based sandbox reuse

The driver creates a fresh container per request. That guarantees a clean starting state and costs 98 ms of container instantiation on every call, which Table II

TABLE V
Hardening checklist for the Docker driver.

Pre-hardening gap	Vector	Status in this release
Writable rootfs	ro_root	fixed: ReadonlyRootfs; confirmed
Default capability set	raw_socket, modprobe	fixed: CapDrop ALL; confirmed
No process cap	fork_bomb	fixed: PidsLimit 256; confirmed
Permissive ptrace	ptrace_agent	denied via capability drop and Yama; custom seccomp pending
Bridge network by default	rfc1918, imds	fail closed: network none; egress policy pending
Oversize artifacts dropped	(none)	upload path pending

shows is the largest phase for Boxed and for raw Docker alike. Reusing a warmed sandbox for compatible follow-up requests would spread that cost, as Oakes et al.'s SOCK [21] does for serverless functions with an LRU cache of paused containers. Compatibility keys naturally on base image, network policy, and resource limits. A reset between uses (kill descendant processes, clear /tmp and /output) is far cheaper than teardown and recreate, and the interface already lets Create return an existing identifier. For a workload that spawns dozens of sandboxes per task this is the largest remaining optimization, and what it costs in isolation strength has to be measured before anyone relies on it.

VIII. Future Work

Four directions follow from the limitations above. A second backend. The interface is designed so that a stronger isolation backend can replace Docker without touching the control plane, and that claim stays untested until a second backend exists; gVisor through runsc on a native Linux host is the cheapest test, and a Firecracker driver the intended one. A custom seccomp profile and egress policy, so that the ptrace denial and the network restriction belong to Boxed and not to the host. Sandbox reuse, measured for both the latency it saves and the isolation it weakens. Native Linux measurement, to separate the cost of the virtual machine from the cost of the substrate. After those, a multi-host scheduler once aggregate concurrency exceeds one machine, and a Wasm/WASI backend once the toolchain runs unmodified language runtimes (Section II).

IX. Conclusion

Boxed is a Docker-based execution substrate for LLM agents that create, use, and destroy sandboxes many times per task. It has a streaming in-sandbox agent, a create that takes one round trip, and a self-hosted control plane, behind an interface meant to admit stronger backends. On the same host, image, and command, the substrate costs 25 ms over calling Docker directly, and the hardened configuration it applies is 63 ms faster than stock Docker defaults. Judged by post-conditions read from the host, the hardened driver stops 12 of 12 escape vectors; the signature-matching method from the earlier version of this work would have reported 2 of them as failures. What is still open is in Section VII: a custom seccomp profile, an egress policy, an artifact upload path, and a second backend that would turn the interface's design intent into a measured property. Implementation, harness, campaign runner, and all raw traces are MIT-licensed at https://github.com/akshayaggarwal99/boxed (tag v0.2.0-paper); bench/analyze/stats.py regenerates every number, table, and figure in this paper from them.

Conflict of Interest

The author maintains Boxed as an open-source side project outside of employment and has no business relationship with, and receives no funding from, Docker, Anthropic, or any vendor in Table I. The raw-Docker baseline, the interleaved runs, and the released harness mitigate the evaluator also being the author.

References

[1] J. Yang, C. E. Jimenez, A. Wettig et al., "SWE-agent: Agent-computer interfaces enable automated software engineering," in NeurIPS, 2024.

[2] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable code actions elicit better LLM agents," in International Conference on Machine Learning (ICML), 2024.

[3] NIST NVD, "CVE-2019-5736: runc container breakout," https://nvd.nist.gov/vuln/detail/CVE-2019-5736, 2019.

[4] X. Lin, L. Lei, Y. Wang, J. Jing, K. Sun, and Q. Zhou, "A measurement study on Linux container security: Attacks and countermeasures," in Annual Computer Security Applications Conference (ACSAC), 2018.

[5] X. Gao, Z. Gu, Z. Li, H. Jamjoom, and C. Wang, "Houdini's escape: Breaking the resource rein of Linux control groups," in ACM SIGSAC Conference on Computer and Communications Security (CCS), 2019.

[6] E2B, "E2B: Open-source secure sandboxes for AI agents," https://e2b.dev, 2024.

[7] Modal Labs, "Modal Sandbox documentation," https://modal.com/docs/guide/sandbox, 2025.

[8] Cloudflare, "Cloudflare Sandbox SDK," https://developers.cloudflare.com/workers/sandbox/, 2025.

[9] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight virtualization for serverless applications," in USENIX NSDI, 2020.

[10] Google, "gVisor: Container runtime sandbox," https://gvisor.dev, 2018.

[11] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "The true cost of containing: A gVisor case study," in USENIX Workshop on Hot Topics in Cloud Computing (HotCloud). USENIX Association, 2019.

[12] Anjali, T. Caraza-Harter, and M. M. Swift, "Blending containers and virtual machines: A study of Firecracker and gVisor," in ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE), 2020, pp. 101–113.

[13] Kata Containers Project, "Kata Containers: Secure containers with the speed of containers," Open Infrastructure Foundation project, https://katacontainers.io, 2018.

[14] F. Manco, C. Lupu, F. Schmidt, J. Mendes, S. Kuenzer, S. Sati, K. Yasukata, C. Raiciu, and F. Huici, "My VM is lighter (and safer) than your container," in SOSP, 2017.

[15] A. Madhavapeddy, R. Mortier, C. Rotsos, D. Scott et al., "Unikernels: Library operating systems for the cloud," in ASPLOS, 2013.

[16] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien, “Bringing the web up to speed with WebAssembly,” in ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2017.

[17] Bytecode Alliance, “WebAssembly system interface (WASI),” https://wasi.dev, 2024.

[18] Daytona, “Daytona: Open-source dev-environment platform,” https://daytona.io, 2024.

[19] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hou, Y. Yang, L. Shirodkar, M. Schmidt, R. Zhang, A. Nguyen, and G. Neubig, “OpenHands: An open platform for AI software developers as generalist agents,” in International Conference on Learning Representations (ICLR), 2025, arXiv:2407.16741.

[20] M. Chen, J. Tworek, H. Jun et al., “Evaluating large language models trained on code,” arXiv preprint arXiv:2107.03374, 2021.

[21] E. Oakes, L. Yang, D. Zhou, K. Houck, T. Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “SOCK: Rapid task provisioning with Serverless-Optimized containers,” in USENIX Annual Technical Conference (USENIX ATC). USENIX Association, 2018, pp. 57–70.